

CSE 4616

Lab 03: IEEE 802.11 and CSMA/CA in OMNeT++

Duration: 1 hour 30 minutes — In-Lab Assessment

Tools: OMNeT++ and INET Framework

Objectives

- Understand the IEEE 802.11 wireless standard and its key parameters.
- Understand the CSMA/CA medium access control protocol and how it prevents collisions in wireless networks.
- Learn how INET models IEEE 802.11 and CSMA/CA through NED submodules and INI parameters.
- Configure and run wireless simulations that vary key 802.11 and CSMA/CA parameters.
- Observe and analyse the effect of parameter changes on network behaviour in Qtenv.

1. Short Theory

1.1 What is IEEE 802.11?

IEEE 802.11 is the family of international standards that define how wireless local area networks (WLANs) operate. It is the technical specification behind what we commonly call **Wi-Fi**. The standard defines:

- The **physical layer (PHY)**: how bits are encoded and transmitted as radio signals (frequency, modulation, data rate).
- The **medium access control (MAC) layer**: the rules governing which device is allowed to transmit at any given moment.

Several amendments to 802.11 exist, each improving speed or range:

Standard	Frequency	Max Data Rate	Common Name
802.11b	2.4 GHz	11 Mbps	Wi-Fi 1
802.11a	5 GHz	54 Mbps	Wi-Fi 2
802.11g	2.4 GHz	54 Mbps	Wi-Fi 3
802.11n	2.4/5 GHz	600 Mbps	Wi-Fi 4
802.11ac	5 GHz	3.5 Gbps	Wi-Fi 5
802.11ax	2.4/5/6 GHz	9.6 Gbps	Wi-Fi 6

Table 1: IEEE 802.11 amendments and their characteristics

In INET, the default wireless model uses **802.11g/n** parameters unless you specify otherwise. The radio medium module `Ieee80211ScalarRadioMedium` models the propagation of 802.11 signals.

1.2 The Hidden Terminal Problem

Before understanding CSMA/CA, it is important to understand *why* a special MAC protocol is needed for wireless networks.

In a wired network, a device can detect a collision immediately because the electrical signal on the cable is always observable. In wireless networks, this is not possible. Consider three devices A, B, and C where:

- A can hear B.
- C can hear B.
- A **cannot** hear C (they are too far apart).

If A and C both transmit at the same time, their signals collide at B, but neither A nor C knows this happened. This is the **hidden terminal problem**, and it is the core motivation for CSMA/CA.

Key insight: In wireless networks, a transmitter *cannot detect its own collisions* the way wired CSMA/CD can. This is why 802.11 uses **collision avoidance** rather than collision detection.

1.3 What is CSMA/CA?

CSMA/CA stands for **Carrier Sense Multiple Access with Collision Avoidance**. It is the MAC protocol used by IEEE 802.11. Its goal is to **reduce the probability of collisions** before they happen, rather than detecting them after the fact.

1.3.1 The Basic CSMA/CA Procedure

When a device wants to transmit a frame:

1. **Carrier Sense:** The device first *listens* to the channel. If the channel is busy (another device is transmitting), it **waits** until the channel becomes idle.
2. **DIFS Wait:** Once the channel is idle, the device waits for a fixed period called **DIFS** (Distributed Inter-Frame Space) before attempting to transmit. This gives higher-priority frames (e.g., acknowledgements) a chance to go first.
3. **Random Backoff:** After the DIFS, the device selects a **random backoff counter** from the range $[0, CW - 1]$, where CW is the **Contention Window** size. The counter counts down only while the channel is idle. If the channel becomes busy, counting pauses until it is idle again.
4. **Transmission:** When the backoff counter reaches zero, the device transmits its frame.
5. **Acknowledgement (ACK):** The receiver, upon successfully receiving the frame, waits for a short period called **SIFS** (Short Inter-Frame Space) and then sends an **ACK** frame back. If the sender does not receive an ACK within a timeout period, it assumes a collision occurred and retransmits.
6. **Exponential Backoff on Collision:** If a collision is detected (no ACK received), the contention window is **doubled** (up to a maximum CW_{max}), and the device tries again with a new random backoff. This is called **Binary Exponential Backoff (BEB)**.

CSMA/CA Timing Summary:

```

[Channel Idle] --> [DIFS] --> [Random Backoff] --> [Transmit Frame]
                                                    |
                                                    [SIFS] --> [ACK]
                        (if no ACK: double CW, retry)

```

1.3.2 Key CSMA/CA Parameters

Parameter	Full Name	Role
DIFS	Distributed Inter-Frame Space	Minimum idle time before any data frame transmission
SIFS	Short Inter-Frame Space	Idle time before ACK/CTS; shorter than DIFS to give ACKs priority
CW _{min}	Minimum Contention Window	Initial range for random backoff counter ($[0, CW_{min} - 1]$)
CW _{max}	Maximum Contention Window	Upper limit after repeated doublings due to collisions
SlotTime	Slot Time	The granularity of the backoff counter; one slot = one count-down step
AIFS	Arbitration Inter-Frame Space	Generalisation of DIFS used in QoS-enabled 802.11e/WMM

Table 2: Key CSMA/CA timing parameters

1.3.3 RTS/CTS: Solving the Hidden Terminal Problem

To address the hidden terminal problem, 802.11 optionally uses **RTS/CTS (Request to Send / Clear to Send)**:

1. Before sending data, the sender transmits a short **RTS** frame to the receiver.
2. The receiver replies with a **CTS** frame, which is heard by all devices in its range (including hidden terminals).
3. All devices that hear the CTS stay silent for the duration of the upcoming data exchange (**virtual carrier sensing** via the Network Allocation Vector, NAV).
4. The sender then transmits its data frame, followed by an ACK.

RTS/CTS adds overhead for small frames but significantly reduces collisions in dense networks. In INET it is controlled by the `rtsThreshold` parameter: frames larger than this threshold trigger RTS/CTS.

2. How INET Models IEEE 802.11 and CSMA/CA

2.1 Key INET Modules

INET implements 802.11 as a stack of modules inside each wireless host. The relevant modules are:

INET Module	Role
Ieee80211ScalarRadioMedium	The shared wireless medium (“air”). Handles signal propagation, path loss, and interference. Declared once in the NED file.
Ieee80211Radio	The physical layer radio inside each wireless host. Transmits and receives frames as radio signals.
Ieee80211Mac	The MAC layer. Implements CSMA/CA, backoff, ACK handling, and RTS/CTS. This is the core of the 802.11 behaviour.
WirelessHost / AccessPoint	Compound modules containing the full 802.11 stack (radio + MAC + IP + transport + apps).

Table 3: Key INET modules for IEEE 802.11 simulation

2.2 The Radio Medium and Path Loss

The Ieee80211ScalarRadioMedium module controls how signals propagate. Key parameters include:

```

1 # Path loss model: how signal strength decreases with distance
2 **.radioMedium.pathLoss.typename = "FreeSpacePathLoss"
3
4 # Background noise level (affects reception sensitivity)
5 **.radioMedium.backgroundNoise.power = -110dBm

```

Listing 1: Configuring the radio medium in omnetpp.ini

2.3 Configuring the 802.11 MAC (CSMA/CA Parameters)

CSMA/CA parameters are set on the wlan[0].mac submodule inside each wireless host. You can override them in the INI file:

```

1 # Contention window minimum (default for 802.11g: 15)
2 **.wlan[0].mac.dcf.channelAccess.cwMin = 15
3
4 # Contention window maximum (default for 802.11g: 1023)
5 **.wlan[0].mac.dcf.channelAccess.cwMax = 1023
6
7 # RTS/CTS threshold: frames larger than this use RTS/CTS (bytes)
8 # Set to a very large value to disable RTS/CTS
9 **.wlan[0].mac.dcf.channelAccess.rtsThreshold = 3000B
10
11 # Maximum number of retransmission attempts before dropping a frame
12 **.wlan[0].mac.dcf.channelAccess.retryLimit = 7
13
14 # Slot time (must match the 802.11 amendment being simulated)
15 **.wlan[0].mac.dcf.channelAccess.slotTime = 9us # 802.11g/a

```

Listing 2: Key CSMA/CA parameters in omnetpp.ini

2.4 Configuring the Physical Layer (Radio Parameters)

```

1 # Transmission power (affects range)
2 **.wlan[0].radio.transmitter.power = 20mW
3
4 # Carrier frequency (2.4 GHz for 802.11b/g, 5 GHz for 802.11a)
5 **.wlan[0].radio.carrierFrequency = 2.4GHz
6
7 # Bitrate / data rate
8 **.wlan[0].radio.transmitter.bitrate = 54Mbps
9
10 # Receiver sensitivity: minimum signal power to decode a frame
11 **.wlan[0].radio.receiver.sensitivity = -85dBm
12
13 # Energy detection threshold: minimum power to sense the channel as busy
14 **.wlan[0].radio.receiver.energyDetection = -85dBm

```

Listing 3: Key PHY parameters in omnetpp.ini

3. Part B: NED File for a Multi-Host Wireless Network

File: WirelessMediumStudy.ned

For experiments, we use an extended network with **four wireless hosts** and one access point. This allows us to observe CSMA/CA contention clearly when multiple devices compete for the same channel.

```

1 package inet.examples.wireless.wiredandwirelesshostswithap;
2
3 import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator;
4 import inet.node.inet.WirelessHost;
5 import inet.node.wireless.AccessPoint;
6 import inet.physicallayer.wireless.ieee80211.packetlevel.
   Ieee80211ScalarRadioMedium;
7 import inet.visualizer.contract.IIntegratedVisualizer;
8
9 network WirelessMediumStudy
10 {
11     submodules:
12         visualizer: <default(firstAvailableOrEmpty("
IntegratedCanvasVisualizer"))>
13             like IIntegratedVisualizer if typename != "" {
14                 parameters:
15                     @display("p=50,50;is=s");
16             }
17         configurator: Ipv4NetworkConfigurator {
18             parameters:
19                 assignDisjunctSubnetAddresses = false;
20                 @display("p=50,150;is=s");
21         }
22         radioMedium: Ieee80211ScalarRadioMedium {
23             parameters:
24                 @display("p=50,250;is=s");
25         }
26         // Four wireless hosts competing for the same channel
27         host1: WirelessHost {

```

```

28         parameters:
29             @display("p=150,100");
30     }
31     host2: WirelessHost {
32         parameters:
33             @display("p=350,100");
34     }
35     host3: WirelessHost {
36         parameters:
37             @display("p=150,300");
38     }
39     host4: WirelessHost {
40         parameters:
41             @display("p=350,300");
42     }
43     accessPoint: AccessPoint {
44         parameters:
45             @display("p=250,200");
46     }
47     // No wired connections needed: all hosts connect wirelessly
48 }

```

Listing 4: WirelessMediumStudy.ned

Note: No connections block is needed. All four hosts associate with the access point wirelessly via the shared radioMedium. The access point acts as the central coordinator, and all host-to-host traffic passes through it.

4. Part C: INI File for 802.11 / CSMA/CA Experiments

File: omnetpp.ini (add as a new [Config] section)

The baseline configuration below sets up four hosts all sending UDP traffic through the access point. All CSMA/CA parameters are at their 802.11g defaults.

```

1  [Config WirelessMediumBaseline]
2  network = WirelessMediumStudy
3  sim-time-limit = 200s
4
5  **.addDefaultRoutes = false
6
7  # --- Application setup -----
8  # All 4 hosts run 2 apps: one echo server, one UDP sender
9  **.*host*.numApps = 2
10
11 **.*host*.app[0].typename = "UdpEchoApp"
12 **.*host*.app[0].localPort = 5000
13
14 **.*host*.app[*].typename = "UdpBasicApp"
15
16 # Traffic pattern: each host sends to one other host via the AP
17 **.host1.app[1].destAddresses = "host3"
18 **.host2.app[1].destAddresses = "host4"
19 **.host3.app[1].destAddresses = "host1"
20 **.host4.app[1].destAddresses = "host2"
21

```

```

22 **.host*.app[1].destPort      = 5000
23 **.host*.app[1].messageLength = 1000B
24 **.host*.app[1].sendInterval  = 0.1s    # high load: 10 pkt/s per host
25 **.host*.app[1].startTime     = 0s
26 **.host*.app[1].stopTime      = 180s
27
28 # --- 802.11 PHY parameters -----
29 **.wlan[0].radio.transmitter.power      = 20mW
30 **.wlan[0].radio.carrierFrequency       = 2.4GHz
31 **.wlan[0].radio.transmitter.bitrate    = 54Mbps
32 **.wlan[0].radio.receiver.sensitivity   = -85dBm
33 **.wlan[0].radio.receiver.energyDetection = -85dBm
34
35 # --- CSMA/CA MAC parameters (802.11g defaults) -----
36 **.wlan[*].mac.dcf.channelAccess.retryLimit = 10
37 **.wlan[*].mac.dcf.channelAccess.cwMin = 15
38 **.wlan[*].mac.dcf.channelAccess.cwMax = 1023
39 # **.wlan[*].mac.dcf.channelAccess.cwMulticast = 15
40 # **.wlan[*].mac.dcf.channelAccess.sifsTime = 10us
41 # **.wlan[*].mac.dcf.channelAccess.slotTime = 20us
42 # **.wlan[*].mac.dcf.channelAccess.difsTime = 10us + 2 * 20us
43 # **.wlan[*].mac.dcf.channelAccess.ackTimeout = 10us + 2 * 20us
44
45 **.wlan[*].mac.rtsThresholdBytes = 3000B
46
47 # --- Result recording -----
48 **.vector-recording = true

```

Listing 5: Baseline INI configuration for WirelessMediumStudy

4.1 Line-by-Line Explanation

- **sendInterval = 0.1s** — Each host sends 10 packets per second, creating a **high-load** scenario with significant channel contention. This makes CSMA/CA behaviour clearly observable.
- **messageLength = 1000B** — 1000-byte packets are large enough to trigger realistic backoff behaviour but small enough to avoid fragmentation.
- **cwMin = 15** — The initial contention window range is $[0, 15]$, so the first backoff is a random integer from 0 to 15 slot times. A smaller CWmin means less initial delay but more collisions when many hosts compete.
- **cwMax = 1023** — After repeated collisions, the window can grow up to 1023, making retransmissions increasingly spaced out (Binary Exponential Backoff).
- **rtsThreshold = 3000B** — Since all packets are 1000B (less than 3000B), RTS/CTS is **disabled** in the baseline. You will enable it in the tasks.
- **retryLimit = 7** — A frame is retransmitted up to 7 times before being dropped. Each retry doubles the contention window.
- **transmitter.power = 20mW** — Transmission power controls the radio range. Reducing this shrinks the coverage area and can create hidden terminal scenarios.

- **bitrate = 54Mbps** — The PHY data rate at which frames are transmitted. Higher bitrate means shorter frame durations, which reduces channel occupancy per packet.

4.2 Understanding the CSMA/CA Timing in This Simulation

With the baseline parameters, we can calculate the expected channel behaviour. For 802.11g:

- **Slot time:** $9\mu s$
- **SIFS:** $16\mu s$
- **DIFS:** $34\mu s$ ($= \text{SIFS} + 2 \times \text{SlotTime}$)
- **Average backoff** (no collision, $\text{CW}_{\min} = 15$): $\frac{15}{2} \times 9\mu s = 67.5\mu s$
- **Frame duration** at 54 Mbps for 1000B: $\approx \frac{8000 \text{ bits}}{54 \times 10^6} \approx 148\mu s$

Total per-frame channel time (approximate, no collision):

DIFS + Backoff + Frame + SIFS + ACK

$= 34 + 67.5 + 148 + 16 + 14 \approx 280\mu s$ per frame.

With 4 hosts each sending 10 frames/s = 40 frames/s total, the channel is occupied approximately $40 \times 280\mu s = 11.2 \text{ ms}$ per second, or about **1.1% load** at 54 Mbps. This is light load, but each task will increase congestion to observe backoff more clearly.

5. Part D: Parameter Reference Tables

5.1 802.11 Amendment Parameter Comparison

Param	802.11b	802.11a	802.11g	802.11n
Frequency	2.4 GHz	5 GHz	2.4 GHz	2.4/5 GHz
Max Rate	11 Mbps	54 Mbps	54 Mbps	600 Mbps
SlotTime	$20\mu s$	$9\mu s$	$9\mu s$	$9\mu s$
SIFS	$10\mu s$	$16\mu s$	$16\mu s$	$16\mu s$
DIFS	$50\mu s$	$34\mu s$	$34\mu s$	$34\mu s$
CW _{min}	31	15	15	15
CW _{max}	1023	1023	1023	1023

Table 4: Default CSMA/CA timing parameters by 802.11 amendment

5.2 Effect of Key CSMA/CA Parameters on Network Behaviour

6. Debugging Tips

- **Wireless host not associating with AP:** Ensure the host is within radio range. Increase `transmitter.power` or reduce the distance in the NED file (`@display` coordinates).
- **No packets received despite association:** Check that `destAddresses` uses the exact module name from the NED file. Verify `destPort` matches `localPort` of the echo app.

Parameter	Increasing effect	Decreasing effect	When to change
CWmin	Fewer collisions, higher initial delay	More collisions, lower delay	High-density networks
CWmax	More aggressive backoff after collisions	Faster recovery but more collisions	Heavily loaded networks
rtsThreshold	RTS/CTS used for more frames (overhead $\uparrow$)	Fewer frames use RTS/CTS	Hidden terminal environments
retryLimit	More retransmission attempts	Frames dropped sooner	Lossy channels
txPower	Larger coverage range	Smaller range, possible hidden terminals	Range vs. interference trade-off
bitrate	Shorter frame durations, less channel time	Longer durations, more channel occupancy	Throughput optimisation

Table 5: Effect of CSMA/CA and PHY parameter changes on network behaviour

- **Parameter path error (e.g., `cwMin` not found):** The exact parameter path varies by INET version. Use the OMNeT++ inspector (right-click module $\rightarrow$ Open Details) to find the exact path in your installation.
- **Simulation runs but no contention visible:** Increase the number of hosts or reduce `sendInterval` to create more load. CSMA/CA backoff is only observable under moderate to high load.
- **RTS/CTS not triggering:** Ensure `rtsThreshold` is set *lower* than `messageLength`. If the threshold exceeds the packet size, RTS/CTS is skipped.